R workshop part 2 - Data visualization

Jan Gačnik

Prepared: 2026-10-26

Visualization in R

Base R includes a function for plotting - `plot()`. Nowadays, plotting in R is most commonly done with `ggplot()` function from the `tidyverse` package.

Benefits:

- The "**g**rammar of **g**raphics" (**gg**) is an intuitive way of building a plot (for most people)
- Layering - add geometries, stats, annotations without complete replotting
- Faceting - splitting one chart into multiple smaller parts
- Modern, publication-ready default themes

Cons:

- Slower than base `plot()`
- Needs "tidy" data for plotting

Vocabulary of `ggplot()`

Data → Aesthetics → Geometry

Data

The data used for the plot

species	flipper_len	body_mass	bill_len
Adelie	181	3750	39.1
Adelie	186	3800	39.5
Adelie	195	3250	40.3
Adelie	NA	NA	NA
Adelie	193	3450	36.7

Aesthetics

The *mapping* of variables to the aesthetics

- x axis
- y axis
- color
- shape
- line type
- …

Geometry

The *geometric object* that will represent the aesthetic mappings

- scatter plot
- regression
- histogram
- box plot
- violin plot
- …

Visualization in `ggplot()`

Let’s run everything in text editor

```
# Loads tidyverse into our environment
library("tidyverse")
```

```
ggplot(
  data = penguins
)
```

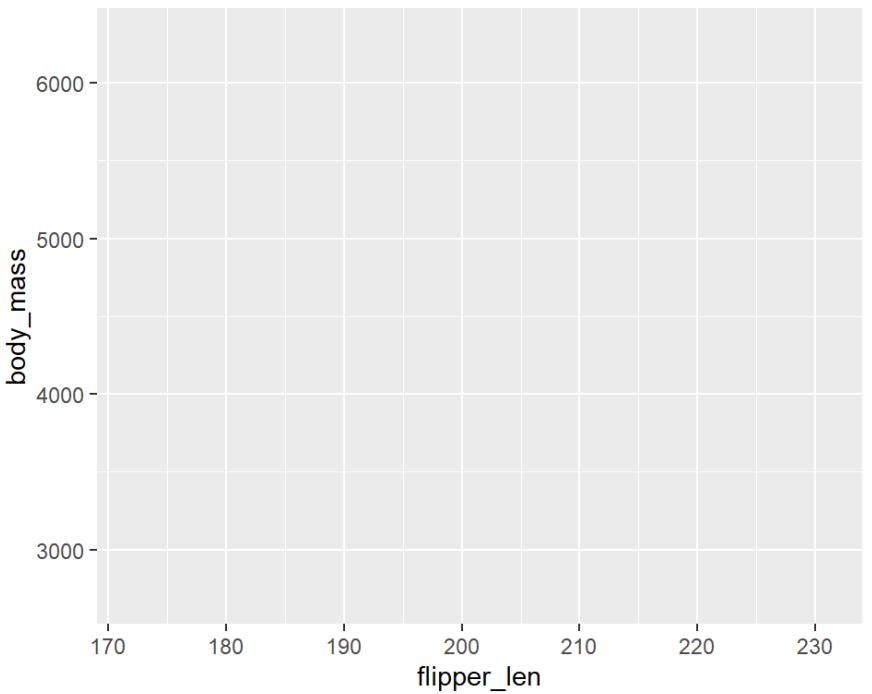
Visualization, axes

Next, we add the mapping argument to our `ggplot()`, with which we define how variables in our dataset are mapped to visual properties (aesthetics)

- Mapping argument is always defined in the `aes()` function

- Let's use `flipper_len` as our `x` aesthetic and `body_mass` as our `y` aesthetic.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_len,
                y = body_mass)
)
```



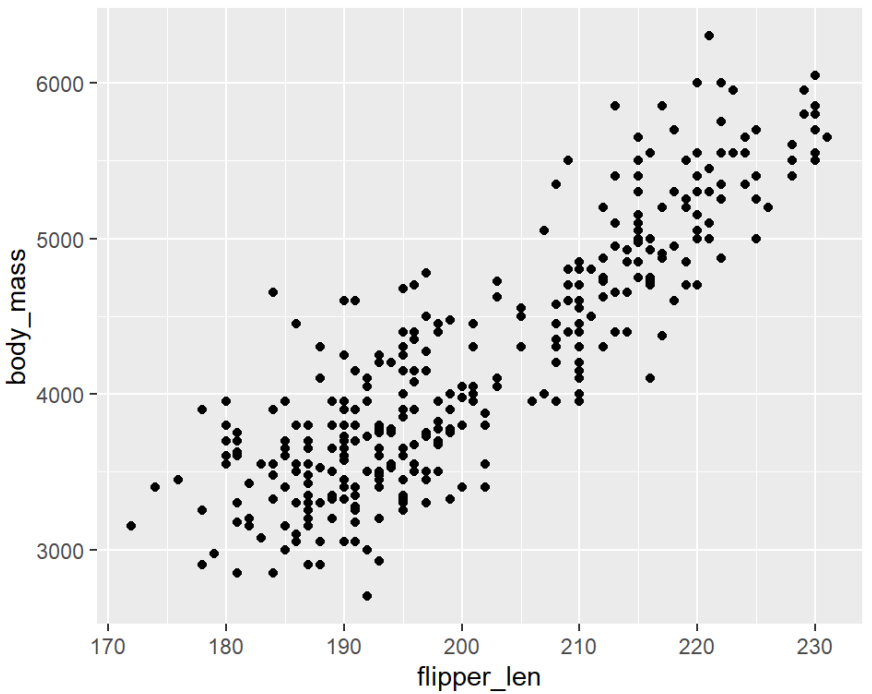
Visualization, geometries

Next, we have to define what kind of **geometry** do we want to plot

- Defined using different `geom_` functions, for example: `geom_point()` (scatter plot), `geom_line()` (linegraph), `geom_boxplot()` (box plot), `geom_violin` (violin plot), etc.

- Let's use `geom_point()` to make our plot a scatter plot:

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_len,
                y = body_mass)
) +
  geom_point()
```

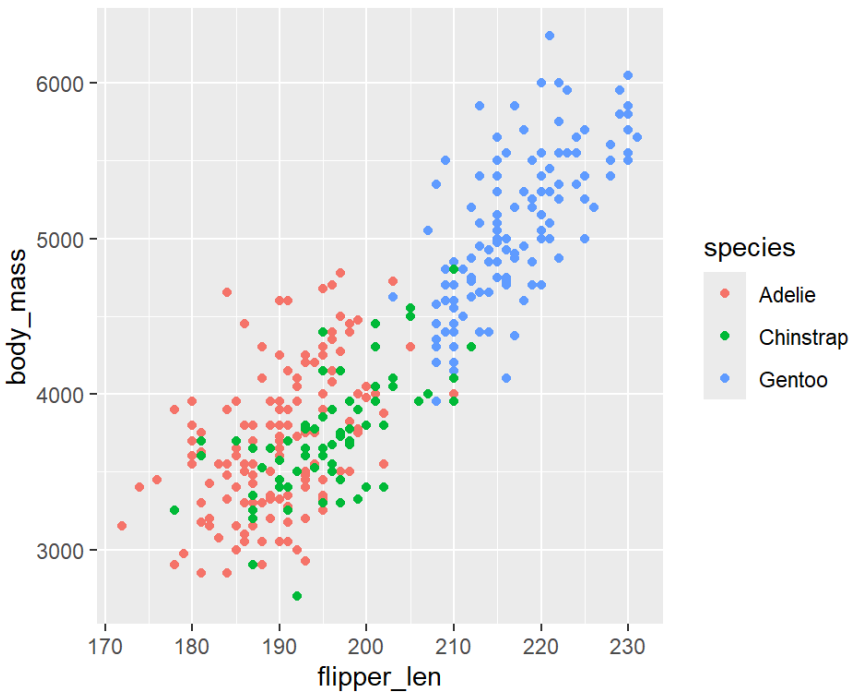


Visualization, color aesthetic

Next, let's make a more informative plot, we would like to see if the relationship between body mass and flipper length differ by *species*

- We need to add another aesthetic for species in `aes()`, for example color:

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_len,
                y = body_mass,
                color = species)
) +
  geom_point()
```

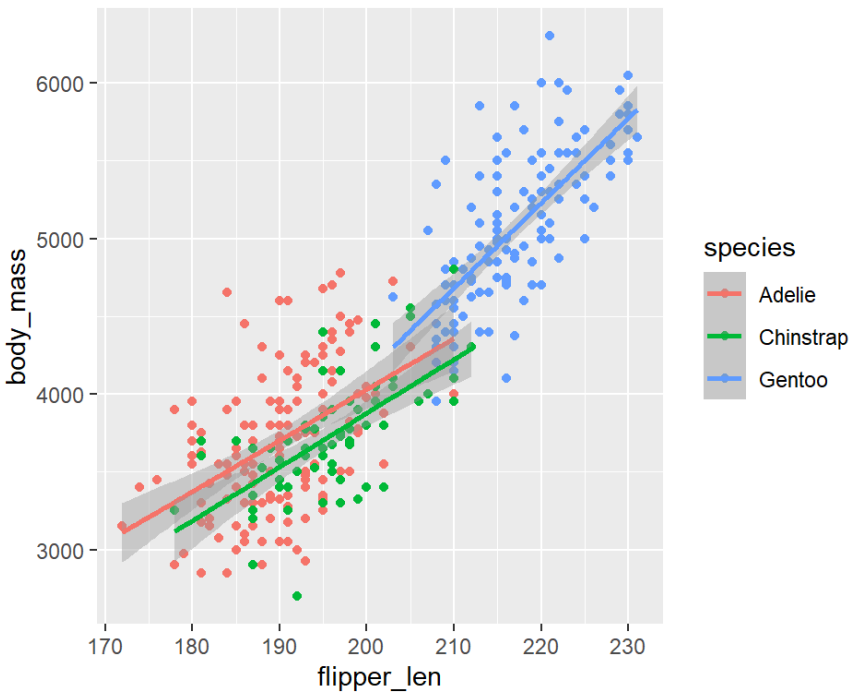


Visualization, linear regression

Next, let's add a *linear regression* to the plot

- Since this is a new geometry, we will add it with `geom_smooth()`. The smoothing we want is a linear model, specified using `method = "lm"`:

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_len,
                y = body_mass,
                color = species)
) +
  geom_point() +
  geom_smooth(method = "lm")
```



? How can we go from *three* linear regressions to *one* for all data **?**

Visualization, aesthetics continued

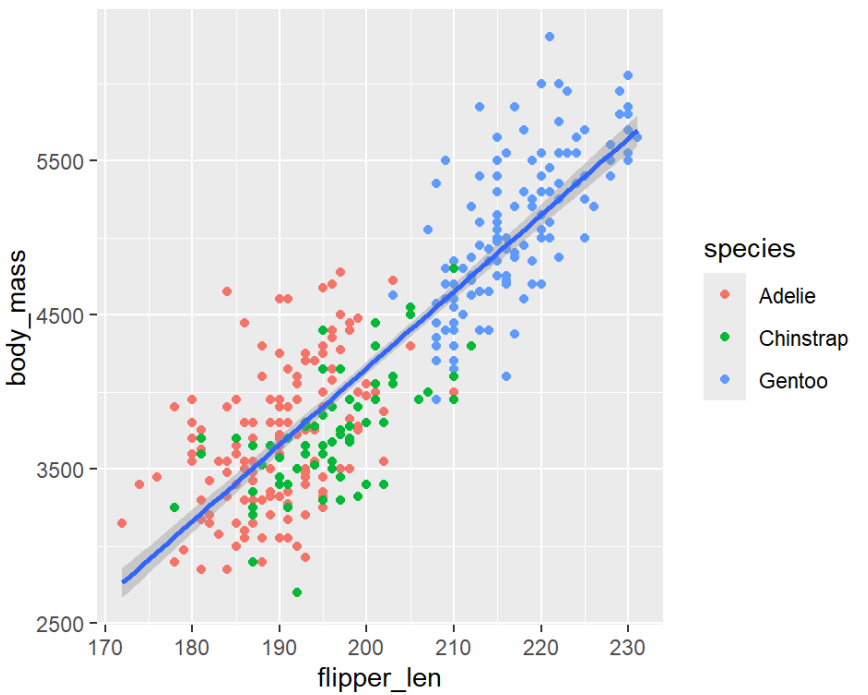
Aesthetics defined in the `ggplot()` function are **global**

Aesthetics defined in geometries are **local**

- We want: 1) points to be colored based on species, 2) no separation of regressions based on species

- So, species variable has to be a local aesthetic in the `geom_point()` mapping:

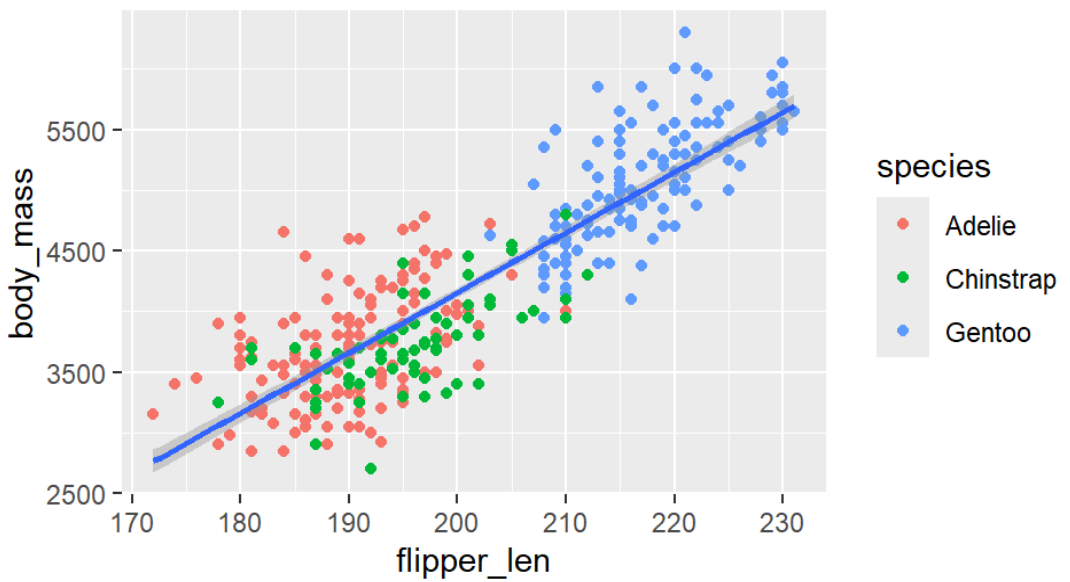
```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_len,
                y = body_mass)
) +
  geom_point(mapping = aes(color = species)) +
  geom_smooth(method = "lm")
```



Visualization, aesthetics continued

As we go on, we will often skip obvious argument names to make the code shorter

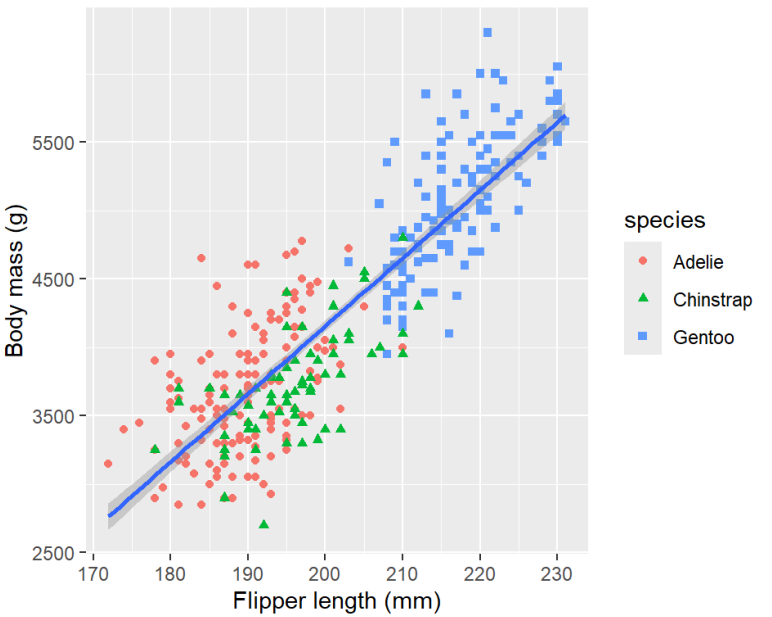
- For example, `data = penguins` becomes `penguins`, and `mapping = aes(…)` becomes `aes(…)`:



Visualization, aesthetics continued

Let's polish the look of the plot a bit

- We can also change the **shape** of the points based on `species` to make things even clearer
  - Where should we add the `shape = species` aesthetic argument?
- Additionally, we can change the labels of axes and legend in the `labs()` function

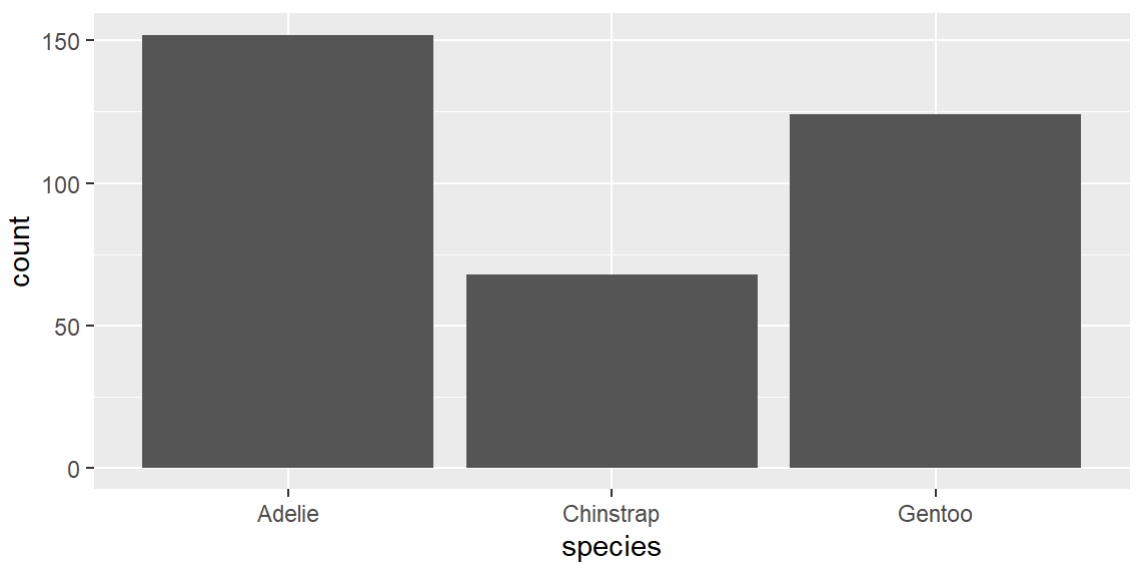


Visualizing distributions

How a distribution is visualized depends on the variable type: **categorical** or **numerical**

- For **categorical** variables, a *bar chart* can be used where the height is proportional to the number of observations of the variable

```
1 ggplot(penguins, aes(x = species)) +
2   geom_bar()
```

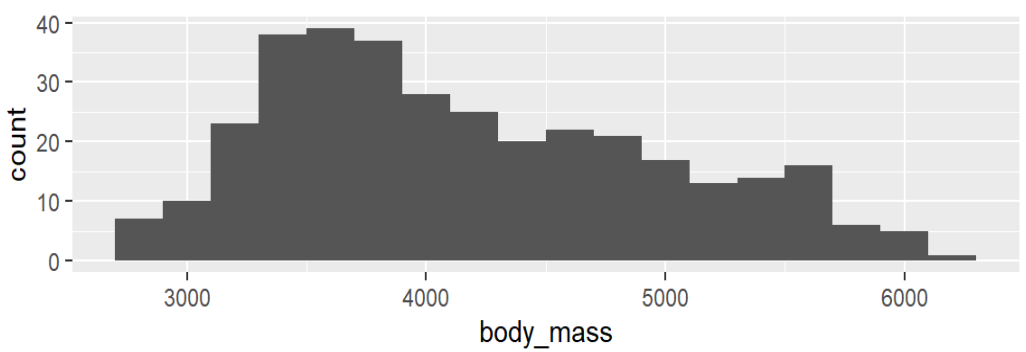


Visualizing distributions

How a distribution is visualized depends on the variable type: **categorical** or **numerical**

- **Numerical** variables can be discrete or continuous. For continuous numerical variables, *histograms* are commonly used (`geom_histogram()`).

```
1 ggplot(penguins, aes(x = body_mass)) +
2   geom_histogram(binwidth = 200)
```

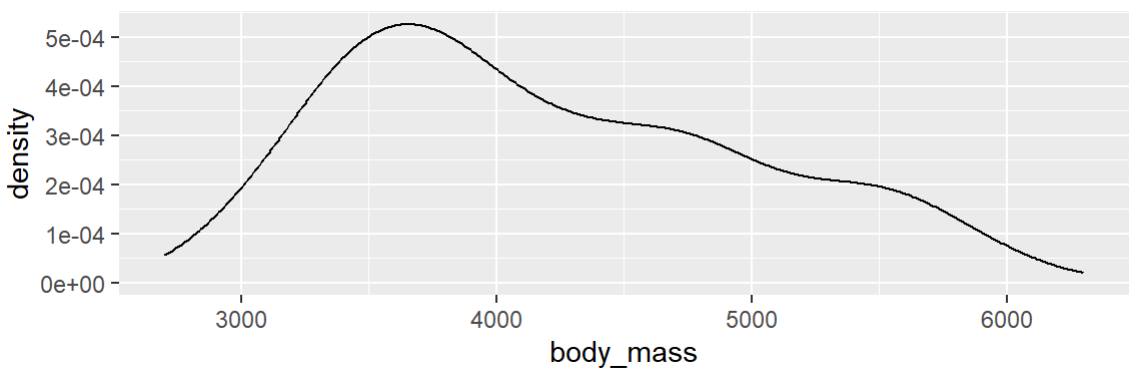


Visualizing distributions

How a distribution is visualized depends on the variable type: **categorical** or **numerical**

- **Numerical** variables can be discrete or continuous. For continuous numerical variables, *histograms* are commonly used (`geom_histogram()`).
- Alternative to a histogram is a *density plot* (`geom_density()`)

```
1 ggplot(penguins, aes(x = body_mass)) +
2   geom_density()
```

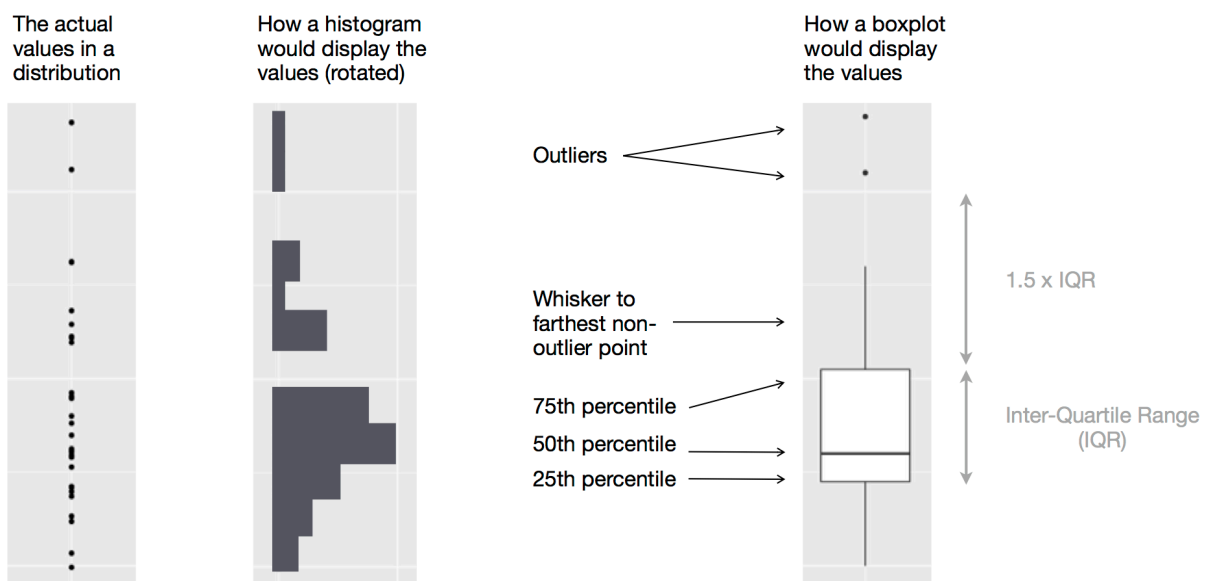


Visualizing relationships

To visualize a relationship, we need at least *two variables* mapped to an aesthetic in a plot (unlike for simple distributions)

For a relationship between a numerical and a categorical variable, a **boxplot** is commonly used

- What even is a boxplot?

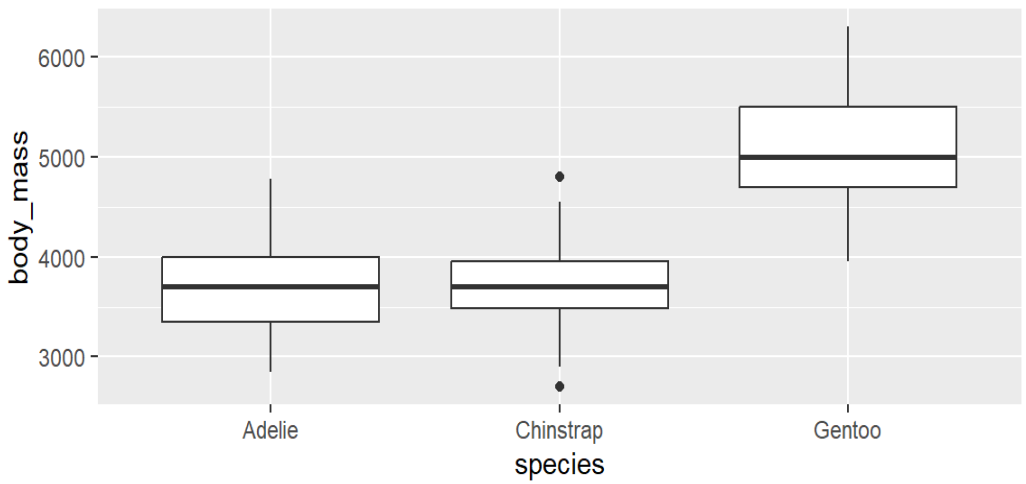


Source: Hadley Wickham, *R for Data Science*

Visualizing relationships: boxplot

Let's make a simple boxplot using the `geom_boxplot()` geometry

```
1 ggplot(penguins, aes(x = species,
2                       y = body_mass)) +
3   geom_boxplot()
```



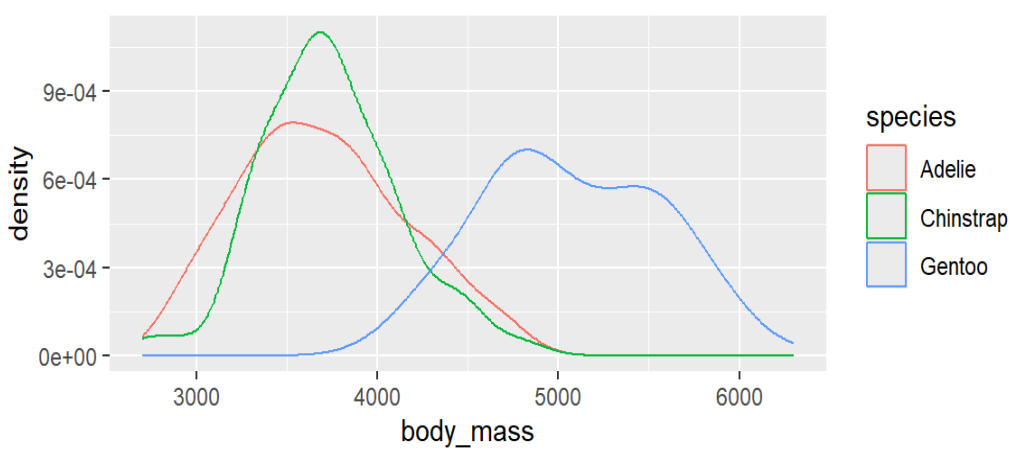

```
1 # We can transform how boxplot looks,
2 # e.g., with outlier.shape = NA or notch = TRUE, etc.
```

Visualizing relationships: density plot

We can again use the density plot using the `geom_density()` geometry

- Remember, 2 aesthetics need to be specified - what happens if only one is specified?
- We can also add the `fill` aesthetic

```
1  ggplot(penguins, aes(x = body_mass,
2                       color = species)) +
3    geom_density()
```

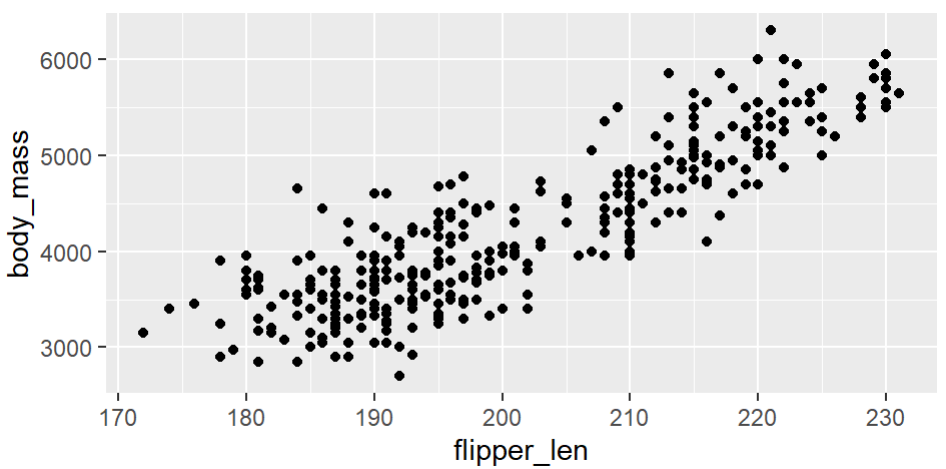


Visualizing relationships: 2 numerical variables

Scatter plot is the most common way of visualizing a relationship between 2 numerical variables, using `geom_point()` geometry

- A revisited example:

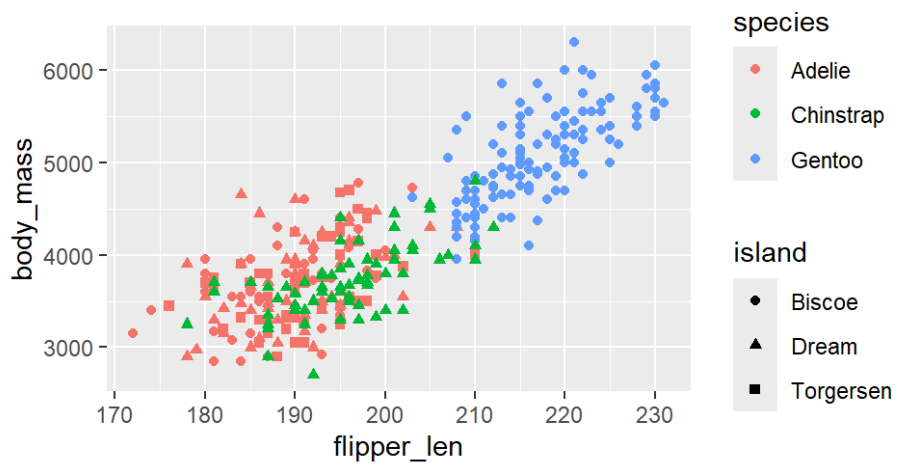
```
1 ggplot(penguins, aes(x = flipper_len,
2                       y = body_mass)) +
3   geom_point()
```



Visualizing relationships: 3 or more numerical variables

We can incorporate more variables into the plot by mapping them to additional aesthetics, for example `color = species` and `shape = island`

```
ggplot(penguins, aes(x = flipper_len,
                     y = body_mass)) +
  geom_point(aes(color = species,
                 shape = island))
```



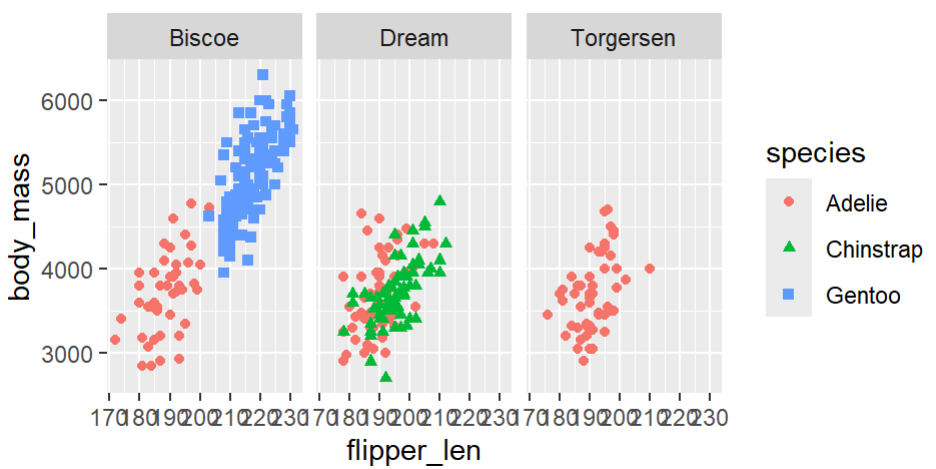
Plot is becoming more and more cluttered, what to do? **FACETING**

Visualizing relationships: faceting

Splitting the plot into multiple subplots - **faceting** - can be very useful to improve readability of plots

- Faceting by a *single variable* uses `facet_wrap()` — specify the variable with `~` followed by its name.
- The faceting variable has to be *categorical*

```
1 ggplot(penguins, aes(x = flipper_len,
2                       y = body_mass)) +
3   geom_point(aes(color = species,
4                  shape = species)) +
5   facet_wrap(~ island)
```



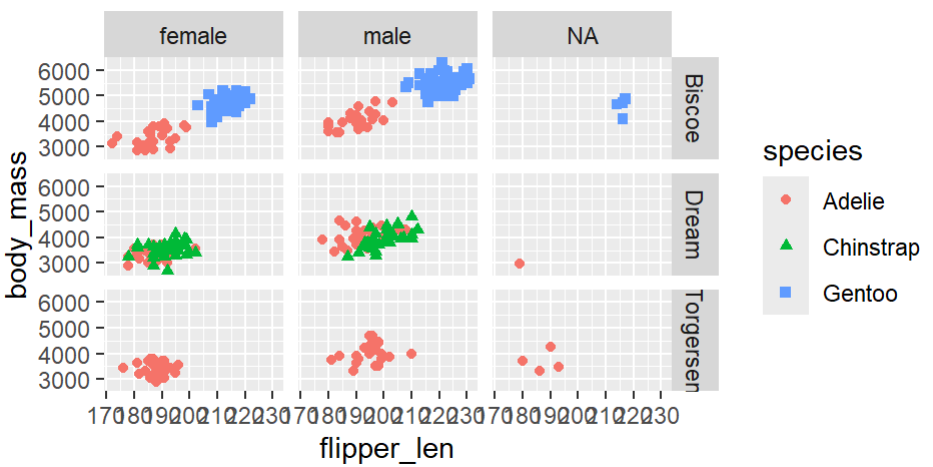
The x axis text overlaps, homework - try solving this by adjusting `theme()` parameters

Visualizing relationships: faceting

Splitting the plot into multiple subplots - **faceting** - can be very useful to improve readability of plots

- Faceting by *two variables* is done using `facet_grid()` — specify the 2 variables using the formula `variable1 ~ variable2`.

```
ggplot(penguins, aes(x = flipper_len,
                     y = body_mass)) +
  geom_point(aes(color = species,
                 shape = species)) +
  facet_grid(island ~ sex)
```



- The `NA` column on the plot indicates entries with missing sex data, we have to **filter** them out
- Filtering part of next presentation - **data transformation**

Saving generated plots

Multiple ways to do it, commonly done using the `ggsave()` function

```
ggsave(filename = "penguin_plot.png")
```

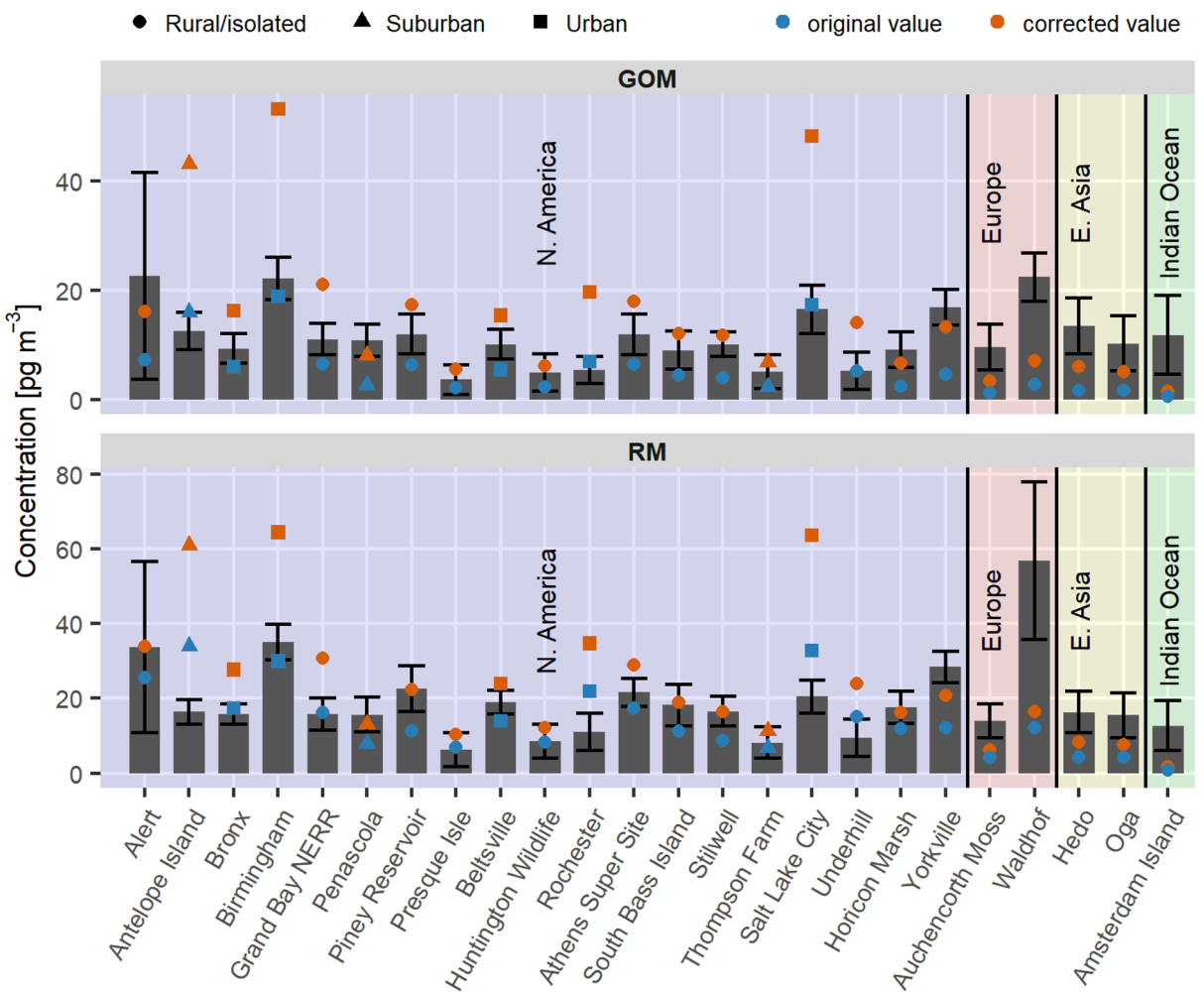
- Specifying `width`, `height`, and `units` arguments will make your plot reproducible

```
ggsave(filename = "penguin_plot.png",
       width = 15, height = 10, units = "cm")
```

- Without specifying dimensions, `ggsave()` will guess the size based on current size of your graphics device

Examples of R plots

Global Hg measurements of reactive Hg, modeled vs measured values Bar plot combined with scatter plot, shapes, lines, and text labels

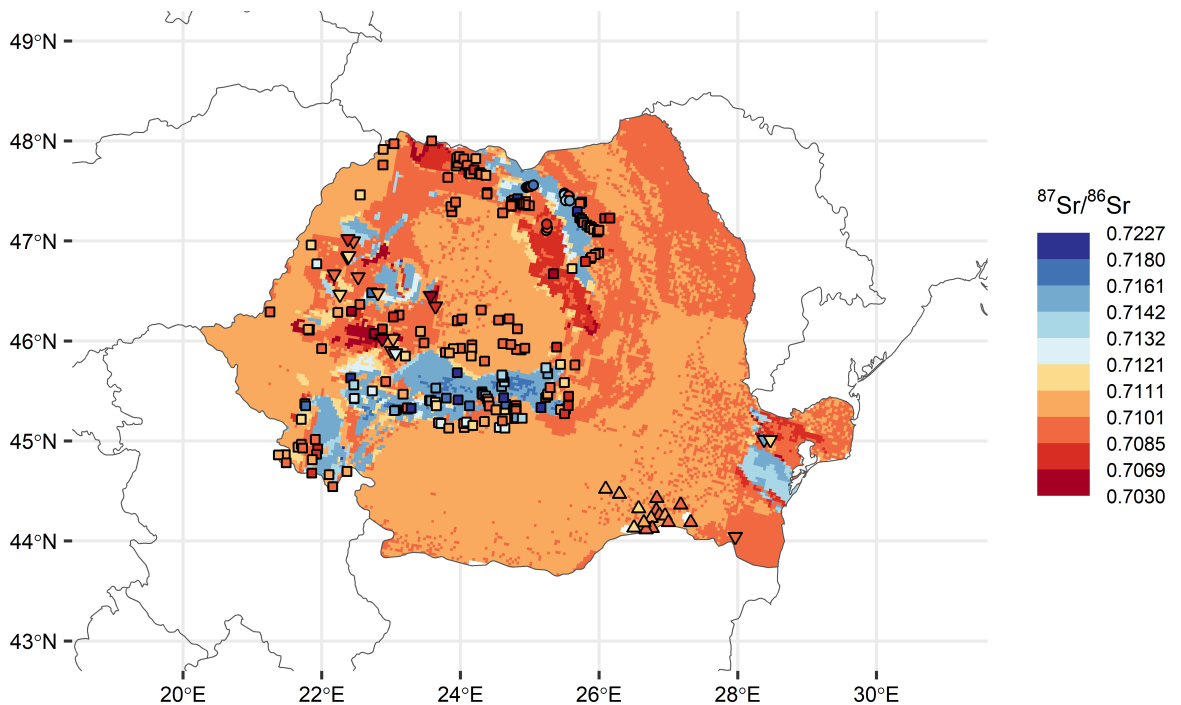


Examples of R plots

Map plots in R

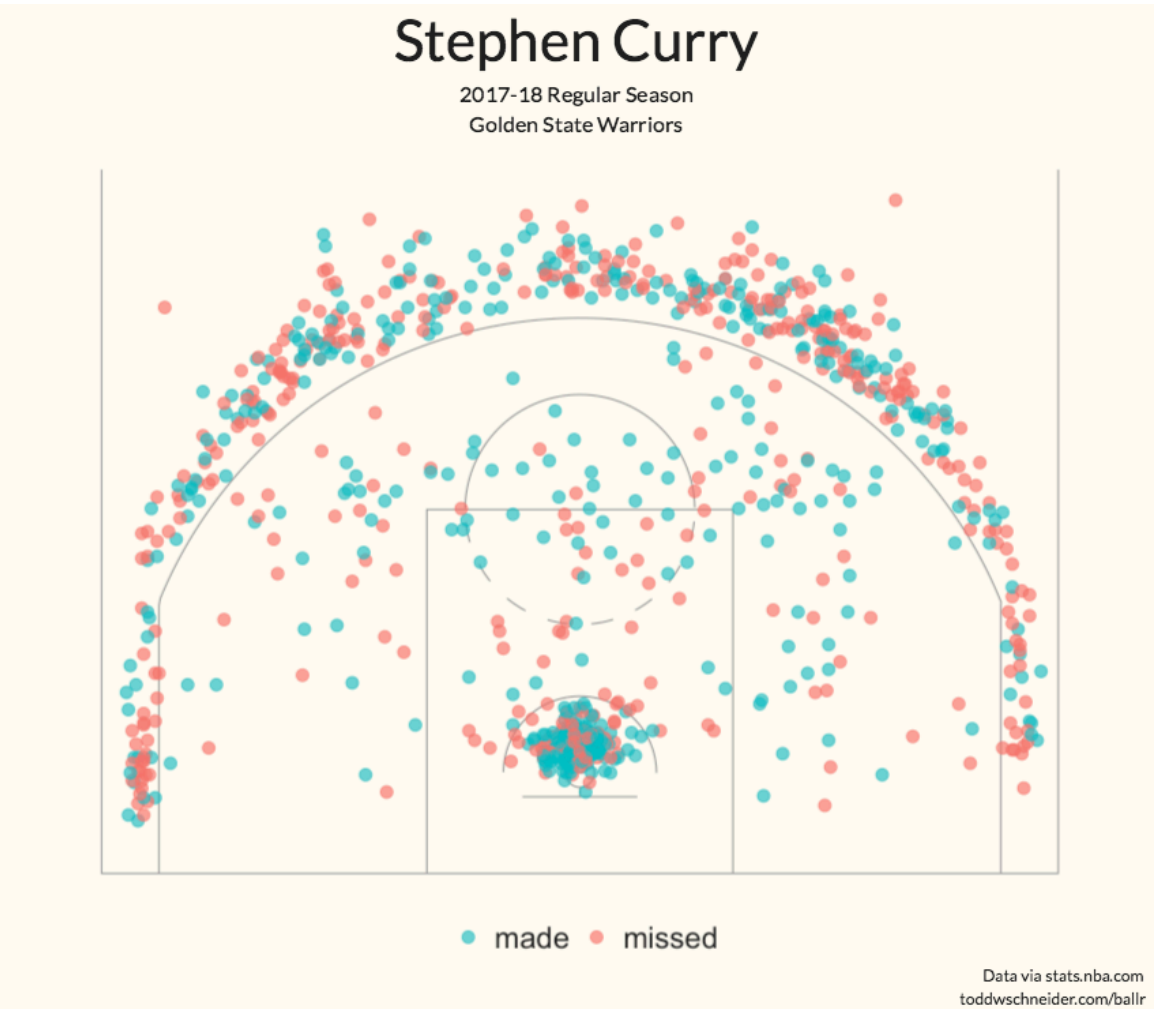
- More control over map appearance, layers, and custom features than in QGIS/ArcGIS
- Requires more coding knowledge and experience to create polished maps

Strontium isotope ratio map of Romania



Examples of R plots

Creativity is the limit with visualization in R



Next up

Plots are only as good as the data behind them. Next, we'll learn to reshape and summarize data so it is ready to plot.

Part 3 - Data transformation with `dplyr`